BRAC UNIVERSITY
Department of Computer Science and Engineering

X

CSE 427: Machine Learning

Sample Semester Final Question

This is a sample question, not a suggestion / extensive practice problem. You should not expect same to same or similar types of questions in the actual final exam. The question pattern, difficulty level, and marks distribution will vary in the exam.

1.

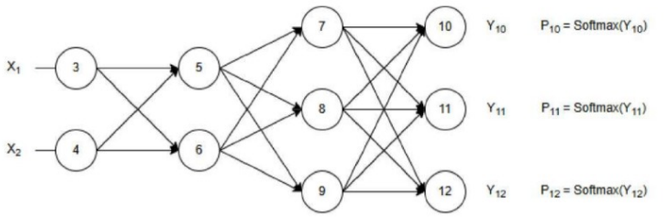


Figure 1

In the network shown in **Figure 1**, the hidden layers use ReLU, and the output layer uses the Softmax activation function. Initial weights of all links are 1. The biases of hidden layers 1, 2, and 3, and the output layer are 2, 3, 5, and 4, respectively. The inputs are $X_1 = 1.5$, $X_2 = 1.75$, target outputs are $P_{10} = 0$, $P_{11} = 1$, $P_{12} = 0$ and learning rate is 0.1. The loss function is defined as Loss = $-\ln(P_{11})$.

a. For this given neural network, show that *[5]*

$$W_{67} = W_{67} - \eta\, Y_7' Y_6 [P_{10} W_{710} + (P_{11} - 1) W_{711} + P_{12} W_{712}]$$

b. Find the updated weight for W_{57} and W_{712}. *[5]*

2. Consider the following unrolled version of an RNN from **Figure 2**.

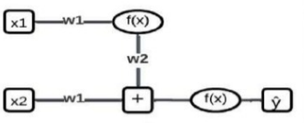


Figure 2

x1	0.5	y	1
x2	2.9	η	0.3
w1	2	Activation	Linear [*f(x) = x*]
w2	-4	Loss	Squared Error

Table 1

a. Find the value of ŷ and calculate the loss using the values given in **Table 1**. *[4]*

b. Find the generic expression for $\frac{dLoss}{dw1}$. From that, explain the vanishing gradient problem of RNN. *[You cannot use any given value to derive the generic expression]* *[4]*

c. Suppose the RNN from Figure 2 is modified by introducing a memory scaling factor α (where $0 \leq \alpha \leq 1$ that controls how much past information is preserved from the previous state). But with the same input sequence passed to this modified RNN, performance is no better, even with larger values of α. Briefly explain the possible limitations of this modified RNN architecture. *[2]*

[Please Turn Over]

Set X

3. You want to translate the Bangla phrase **"চলো আর্জেন্টিনা"** into the Spanish phrase **"Vamos Argentina"** . You have decided to use a Transformer architecture to build this tool. Before the translation process can begin, you need to convert these words into numerical representations. To achieve this, you trained a small neural network to act as your embedding layer. This network has two output units, both using the Sigmoid activation function, producing a 2-dimensional embedding for each word. The weight matrix of the network is given below (**Table 2**). The vocabulary is defined as: Vocabulary = {"চলো", "আর্জেন্টিনা", "দমসু"}.

1.9	-0.5	0.9
-0.7	1.3	-0.2

Table 2

Q		K		V	
0.3	-0.2	-0.4	-0.6	0.3	0.4
-0.1	0.9	-0.3	-0.5	0.9	0.1

Table 3

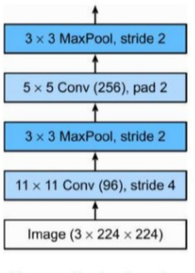
a. Find the word embedding for the word "চলো" and "আর্জেন্টিনা". *[2]*

b. Find the embedding value for the word "চলো" and "আর্জেন্টিনা" after positional encoding. *[3]*

c. The Weight, Query, Key, and Value for the encoder are given above (**Table 3**). Find the final output of the encoder after applying attention between the words. *[5]*

4.

a. The image on the right illustrates the first few layers of AlexNet, the first modern CNN architecture. The input image has 3 channels and dimensions of 224×224, and the first convolutional layer has 96 filters. The max pooling layers don't change the number of channels.
Find out the dimensions (channel_count × width × height) of the input and output matrix of each layer. Stride and pad are set to 1 and 2, respectively, in the 5×5 convolutional layer. All the other layers have pad = 0. *[5]*



b.

	Channel 1		Channel 2	
Image 1	2	4	1	3
	6	8	5	7
Image 2	10	12	9	11
	4	6	3	5
Image 3	8	10	7	9
	12	2	11	1

Consider the batch of three 2×2 images, each having 2 channels. Apply Batch Normalization to this batch and compute the normalized value for each cell in each image. Use this normalization formula:

$$normalized_value = \frac{current_value - min_value}{max_value - min_value}$$

where, following the convention, min_value and max_value are computed per channel, across all three images. *[3]*

c. Briefly explain the difference between Batch Normalization and Instance Normalization. *[2]*